Installation — one-click setup, cross-platform (2026-07-05)

ELI v2.1.11

July 2026

Contents

Installation — one-click setup, cross-platform (2026-07-05)

> **Updated for v2.1.11 (July 2026).** The primary install path is now the prebuilt installers on GitHub Releases — the Windows Setup.exe and Linux AppImage are CI-launch-tested; the macOS dmg is built on a Mac, best-effort (not CI-verified) with first-boot GPU (CUDA/Vulkan/Metal) and starter-model offers; data lives in a per-user `ELI_v2` folder that survives upgrades. Source installs below remain fully supported.

One command per platform sets up Python deps, the **full SQLite architecture** (blank slate — schema only, no personal data), the **nomic embedder**, and the default voice.

Platform	Command	GPU
Linux	`bash install.sh` (`--install-cuda` to also fetch the toolkit)	CUDA
macOS	`bash install.sh`	Metal (no CUDA)
Windows	`install.bat` / `install.bat /cuda` (→ `install.ps1`)	CUDA (winget toolkit)
Android	`bash scripts/install_android.sh`	CPU only (headless)
Portable tarball	`./INSTALL_ELI.sh` then `./RUN_ELI.sh --with-github-assets`	Same as host

CUDA toolkit option (`--install-cuda` / `/cuda`)

For non-technical users with an NVIDIA GPU but no toolkit. Best-effort, never fatal: - **Linux:** tries no-sudo `pip nvidia-cuda-nvcc-cu12` (exposes nvcc via `CUDACXX`), then the system package manager (`apt/dnf/pacman`, if sudo is available), then prints the manual step — then source-rebuilds llama-cpp with `-DGGML_CUDA=on`. - **Windows:** `winget install Nvidia.CUDA`, then rebuilds llama-cpp. - **macOS/Android:** N/A (Metal /

CPU). The default install already uses prebuilt CUDA wheels (no toolkit needed); the option only matters when those don't match the user's CUDA or a source build is required.

What `bash install.sh` does

1. Detects Python (3.10+) and OS; creates `.venv`; upgrades pip/setuptools/wheel.
2. Installs **PyTorch** (CUDA 12.1 / CPU / macOS-MPS per flags/OS).
3. Installs **llama-cpp-python** with GPU acceleration (CUDA wheel index / Metal / CPU) — then **verifies** `llama_supports_gpu_offload()` and, if it landed CPU-only, prints the exact CUDA-rebuild command (closes the silent-CPU-wheel trap).
4. Installs the ELI package (`[full]`) + all remaining dependencies from the **frozen** `requirements.lock.txt` (exact known-good versions; reproducible).
5. Seeds `config/settings.json` from the template (offline-by-default, wizard on) — never overwrites an existing config.
6. Runs `python -m eli.core.init_data` — creates **every** SQLite store/table (`user.sqlite3`, `system_index.sqlite3`, `coding_memory.sqlite3`, `agent.sqlite3`) with **zero personal memories/profile/history** (blank slate). System inventory (installed apps, `$PATH` binaries) is scanned once so "open Firefox" works — that is machine environment data, not user memories.
7. Verifies `import eli`, the GUI entry, and the `eli` console script.
8. Fetches `nomic-embed-text-v1.5.Q4_K_M.gguf` → `models/embeddings/` (~80 MiB, required for memory/RAG) via `python -m eli.core.model_download --aux`.
9. Fetches **voice weights**: Piper `en_US-amy-medium` + faster-whisper `small.en` via `python -m eli.runtime.voice_assets` (idempotent; skipped if already present).
10. Optionally downloads a **chat GGUF** (wizard / `--model=` / `--auto`).
11. Prints how to launch + how to add more models.

Three local asset layers (not in git)

Asset	Path	Size (typical)	When fetched
Embedder (required)	`models/embeddings/nomic-embed-text-v1.5.Q4_K_M.gguf`	~80 MiB	`install.sh` / wizard / `--aux`
Chat model (pick one+)	`models/*.gguf`	0.6–8+ GiB	Wizard / `model_download` / asset pack
Voice (default amy)	`models/tts/piper/` + `tts_piper/piper/`	~60 MiB Piper + ~464 MiB whisper	`voice_assets` / asset pack

GitHub Release tag `local-assets-v2.1` mirrors embedder + starter chat GGUFs + cleared Piper voices. Restore: `./RUN_ELI.sh --with-github-assets`. Voices excluded from auto-restore: ryan, lessac, cori — see `models/MODEL_LICENSES.md`.

Flags

- `--cpu-only` — no CUDA (CPU torch + CPU llama-cpp).
- `--latest` — use `requirements.txt` version ranges instead of the frozen lock.
- `--skip-torch` — leave an existing torch in place.
- `--no-model` — skip chat-model offer; embedder still fetched unless you disable network.

Files

- `install.sh` (Linux/macOS), `install.bat` / `install.ps1` (Windows).
- `requirements.lock.txt` — frozen exact versions (excludes torch/llama-cpp, which install via their CUDA indices).
- `pyproject.toml` — package metadata + `[project.scripts]` (`eli` → `eli.gui.app:main`).

Launch

`./eli.sh` or `source .venv/bin/activate && eli`. First run shows the **setup wizard** if no chat GGUF is present. Wizard also verifies embedder + voice and can fetch them.

Chat model: `python -m eli.core.model_download --auto` (or `--list`, or a named model). ELI stays offline by default; downloads are deliberate one-time actions.